

Framework Topology Projection Module Specification

Whole-Path Orientation-Aware Periodic Framework Graphs for mdstats Stage 2

mdstats

2026-07-13

Contents

Purpose and status	3
Design motives	3
Responsibility boundaries	4
Owned by framework_topology.py	4
Not owned by this module	4
Terminology	5
Public API overview	5
Framework atom roles	6
Role semantics	6
VERTEX	6
LINKER	6
SPECTATOR	6
EXCLUDED	6
Role precedence	6
Accepted path rules	6
FrameworkPathRule	6
Direct projected edges	7
Rule constraints	7
Symbol convenience constructor	7
Framework mapping	8
FrameworkMapping	8
Mapping constraints	8
Derived properties	8
Symbol convenience constructor	8
Serialization	9
Resolved roles	9
ResolvedFrameworkRoles	9
Constraints	9
Role resolution function	9
Projection safety options	10
FrameworkProjectionOptions	10
Constraints	10
Periodic atomic-path convention	10
Projected edge identity	11
FrameworkEdgeKey	11
Key constraints	11
Orientation canonicalization	11
Parallel edges	11
Projected edge provenance	12
FrameworkEdgePath	12
Path constraints	12
Oriented edge traversal view	12

OrientedFrameworkEdgePath	12
Projection report	13
FrameworkProjectionReport	13
Derived diagnostics	13
Framework topology object	13
FrameworkTopology	13
Array shapes	14
Topology constraints	14
Degree convention	14
Components	14
Convenience interface	14
Role-constrained projection algorithm	15
Step 1: validate and resolve roles	15
Step 2: construct oriented periodic adjacency	15
Step 3: precompute accepted rule prefixes	15
Step 4: traverse the lifted atomic graph	16
Step 5: accept terminal paths	16
Step 6: canonicalize path orientation	16
Step 7: deduplicate exact path records	16
Step 8: normalize the projected periodic gauge	17
Step 9: calculate graph observables and identity	17
Pseudocode	17
Complexity	18
Structural identity and digests	19
Mapping digest	19
Graph digest	19
Mapping-aware topology digest	19
Validation API	20
FrameworkValidationRules	20
FrameworkValidationIssue	20
FrameworkValidationReport	20
Validation function	21
Primary build function	21
Inputs	21
state	21
mapping	21
validation_rules	21
options	21
Output	22
Failure modes	22
LTA zeolite reference mapping	22
Edge cases and required behavior	23
Unmapped active species	23
Same species in several roles	23
Spectator contacts	23
Intermediate framework vertices	23
Terminal and dangling linkers	23
Branching linkers	23
Linker-only cycles	23
Parallel paths	23
Self-image edges	23
Zero-shift self-return	23

Repeated canonical atoms in different images	24
Asymmetric linker sequences	24
Overlapping rules	24
Excessive path growth	24
Disconnected or broken frameworks	24
Strain and cell rotation	24
Atomic connectivity model changes	24
Pseudoelements and nonstandard atomic numbers	24
Serialization contract	24
NetworkX conversion	25
Implemented work plan	25
F2.1 - role and mapping foundation	25
F2.2 - periodic path projection	25
F2.3 - projected graph canonicalization	26
F2.4 - diagnostics and validation	26
F2.5 - interoperability and documentation	27
F2.6 - Na-LTA integration	27
Test matrix	27
Implementation status	28
Acceptance criteria for the module	28
Future extensions	28
Final architectural invariant	29

Purpose and status

This document specifies the implemented public module

`mdstats/analysis/framework_topology.py`

for `mdstats` Stage 2. The orientation repair is implemented in `mdstats 0.15.0`. The API, data contracts, algorithms, invariants, validation semantics, and implementation sequence in this document are normative for this release.

The module converts one immutable atom-level connectivity state into one role-aware periodic decorated framework graph. Mapping and topology identity use `mdstats.framework-mapping.v2` and `mdstats.framework-topology.v2`. Version 2 repairs asymmetric linker semantics by coupling endpoint species and linker order under complete path reversal. It answers:

Which atoms are structural framework vertices, which atoms are internal linkers, and which accepted atomic paths define the abstract framework edges?

The central transformation is

$$\boxed{G_{\text{atomic}} \xrightarrow{\text{role assignment and path contraction}} G_{\text{framework}}}$$

For an aluminosilicate zeolite,



but the projected edge retains the complete atomic path, linker identity, chemical rule, and periodic translation.

This module does not enumerate rings. It produces the authoritative projected graph that later ring modules consume.

Design motives

Atom-level connectivity is necessary but not sufficient for framework and ring analysis. A chemically meaningful framework may contain:

- retained tetrahedral atoms such as Si and Al;
- bridging atoms such as O or S;

- extra-framework cations such as Na, Li, or K;
- defects, terminal linkers, branches, and disconnected fragments;
- multiple chemically distinct paths between the same retained vertices;
- edges crossing periodic boundaries.

Treating every atom as an equivalent graph vertex makes ring definitions depend on chemical representation. For example, a six-membered tetrahedral ring appears as a twelve-edge alternating T-O cycle in the atom graph but as a six-edge T-site cycle in the projected framework graph.

The projection layer therefore separates four roles:

VERTEX
LINKER
SPECTATOR
EXCLUDED
.

The design is explicit rather than heuristic. Species defaults may be overridden by atom index, linker sequences are accepted only through declared rules, and validation reports deviations without silently repairing the graph.

Responsibility boundaries

Owned by `framework_topology.py`

The module owns:

- framework role definitions;
- normalized species-role mappings;
- atom-index role overrides;
- exact accepted linker-path rules;
- role resolution for one connectivity state;
- bounded role-constrained path discovery;
- contraction of accepted paths into projected edges;
- atomic-path and linker provenance;
- periodic translation composition;
- projected periodic gauge normalization;
- deterministic multigraph ordering;
- projected degree and component arrays;
- stable serialization and structural digests;
- material-specific topology validation;
- projection diagnostics.

Not owned by this module

The module must not:

- infer atomic bonds from coordinates;
- modify `ConnectivityScope`;
- compare multiple frames or build topology catalogs;
- smooth transient connectivity changes;
- enumerate primitive rings;
- calculate ring centers, normals, radii, or apertures;
- classify ring sites, cages, adsorption, or occupancy;
- assign dynamic geometric regions;
- infer scientific topology from plotting metadata.

The authoritative dependency direction is

```

AtomisticFrameCollection
    |
    v
AtomicConnectivityState
    |
    v

```

```

FrameworkMapping
  |
  v
FrameworkTopology
  |
  +--> topology_catalog.py
  +--> primitive_ring.py
  +--> framework_visualization_adapter

```

framework_topology.py consumes connectivity. Atomic connectivity may use the exact S4 dense, cell-list, or cached execution path, but framework topology never calls the neighbor subsystem, manages a cache, or creates an independent bond definition.

Terminology

- **Atomic graph:** the periodic atom-level graph in AtomicConnectivityState.
- **Framework vertex:** an atom retained as a node in the projected graph.
- **Linker:** an atom permitted only inside a contracted framework path.
- **Spectator:** an atom that remains scientifically relevant but cannot affect projected framework connectivity.
- **Excluded atom:** an atom ignored by the current framework projection.
- **Atomic path:** an ordered path in the lifted periodic atomic graph.
- **Projected edge:** one accepted vertex-to-vertex atomic path represented as a decorated framework edge.
- **Parallel projected edges:** distinct accepted paths with the same projected endpoint atom indices.
- **Self-image edge:** an edge from a framework vertex to a periodic image of the same canonical vertex.
- **Raw projected shift:** the accumulated path translation before framework gauge normalization.
- **Canonical projected shift:** the deterministic translation stored in the final framework edge key.
- **Validation:** optional material-specific checks that do not alter topology.

Public API overview

mdstats 0.15.0 exports the following symbols from `mdstats.analysis` and re-exports them from `mdstats`:

```

CANONICAL_FRAMEWORK_MAPPING_SCHEMA
CANONICAL_FRAMEWORK_TOPOLOGY_SCHEMA
FRAMEWORK_DIGEST_ALGORITHM

```

```

FrameworkAtomRole
FrameworkPathRule
FrameworkMapping
ResolvedFrameworkRoles
FrameworkProjectionOptions

```

```

FrameworkEdgeKey
FrameworkEdgePath
FrameworkProjectionReport
FrameworkTopology

```

```

FrameworkValidationRules
FrameworkValidationIssue
FrameworkValidationReport

```

```

resolve_framework_roles
build_framework_topology
validate_framework_topology

```

The module also defines the following public exceptions:

```

FrameworkTopologyError
FrameworkMappingError
FrameworkProjectionError

```

FrameworkComplexityError
FrameworkValidationError

All public identity-bearing objects must be immutable after construction.

Framework atom roles

```
from enum import Enum
```

```
class FrameworkAtomRole(str, Enum):  
    VERTEX = "vertex"  
    LINKER = "linker"  
    SPECTATOR = "spectator"  
    EXCLUDED = "excluded"
```

Role semantics

VERTEX

A VERTEX atom appears exactly once as a canonical node in the projected graph, even when it is isolated or belongs to a disconnected fragment.

LINKER

A LINKER atom may occur only as an internal atom of a projected path. It must never become a projected node.

SPECTATOR

A SPECTATOR atom is intentionally outside the framework. Atomic edges touching spectators remain present in the source connectivity state but cannot enter a projected path.

Examples include Na, Li, K, Rb, guest molecules, and extra-framework ions.

EXCLUDED

An EXCLUDED atom is ignored by the projection. It differs from SPECTATOR only in scientific intent and provenance. Neither role may alter the projected graph.

Role precedence

For active atom index i with atomic number Z_i , role resolution is

$$R_i = \begin{cases} R_i^{\text{override}}, & i \in \mathcal{O}, \\ R_{Z_i}^{\text{species}}, & Z_i \in \mathcal{S}, \\ R_{\text{unmapped}}, R_{\text{unmapped}} \neq \text{None}, \\ \text{error}, & \text{otherwise} . \end{cases}$$

Atom-index overrides always win. This is required for cases such as:

- framework Al versus extra-framework Al;
- bridging O versus terminal or molecular O;
- framework-bound metal atoms versus mobile ions;
- several materials containing the same chemical species.

Accepted path rules

FrameworkPathRule

```
@dataclass(frozen=True, slots=True)  
class FrameworkPathRule:  
    rule_id: str  
    linker_atomic_numbers: tuple[int, ...]  
    endpoint_atomic_numbers: tuple[  
        int | None,
```

```

    int | None,
] = (None, None)
edge_kind: str = "framework"

```

A path rule declares one complete chemical path pattern

$$\Sigma = (Z_{\text{source}}, Z_{\ell_1}, \dots, Z_{\ell_m}, Z_{\text{target}}).$$

None at an endpoint is a wildcard matching any atom currently resolved as VERTEX. The linker sequence remains exact and contains no wildcard entries.

The endpoint species and linker order are one coupled object. Reversal equivalence applies to the complete path:

$$\Sigma^{-1} = (Z_{\text{target}}, Z_{\ell_m}, \dots, Z_{\ell_1}, Z_{\text{source}}).$$

Therefore,

$$A - O - S - B \equiv B - S - O - A$$

but

$$A - O - S - B \not\equiv A - S - O - B.$$

This distinction is required for asymmetric bridges. The projected adjacency is still undirected; only the edge decoration is orientation-aware.

For deterministic identity, the implementation compares the declared complete signature and its complete reverse, then stores the lexicographically smaller representation. Endpoint and linker canonicalization must never be performed independently.

Direct projected edges

A direct vertex-vertex atomic edge is accepted only by a rule with

```
linker_atomic_numbers=()
```

There is no implicit direct-edge rule. Endpoint restrictions and complete reversal semantics apply in the same way as for linker-containing paths.

Rule constraints

Each rule must satisfy:

- rule_id is nonempty and unique within one mapping;
- edge_kind is nonempty;
- linker atomic numbers are positive standard atomic numbers;
- endpoint_atomic_numbers contains exactly two entries;
- each endpoint entry is either None or a positive standard atomic number;
- the complete signature is canonical modulo whole-path reversal;
- no two rules have overlapping accepted complete signatures.

Two rules overlap when at least one oriented form has the same linker sequence and compatible endpoints, where None is a wildcard. Declaration order must never resolve an overlap.

One rule represents one reversal-equivalence class. Multiple chemically distinct endpoint/linker patterns should use multiple rules with distinct rule_id values.

Symbol convenience constructor

```

FrameworkPathRule.from_symbols(
    rule_id: str,
    linker_symbols: Iterable[str],
    *,
    endpoint_symbols: tuple[
        str | None,

```

```

        str | None,
    ] | None = None,
    edge_kind: str = "framework",
) -> FrameworkPathRule

```

endpoint_symbols=None is equivalent to (None, None). The returned object stores normalized atomic numbers and a complete reversal-canonical signature.

Example:

```

FrameworkPathRule.from_symbols(
    "Si-O-S-Al",
    ("O", "S"),
    endpoint_symbols=("Si", "Al"),
    edge_kind="asymmetric_bridge",
)

```

This accepts Si-O-S-Al and Al-S-O-Si, but rejects Si-S-O-Al.

Framework mapping

FrameworkMapping

```

@dataclass(frozen=True, slots=True)
class FrameworkMapping:
    species_roles: Mapping[int, FrameworkAtomRole]
    atom_role_overrides: Mapping[int, FrameworkAtomRole] = field(
        default_factory=dict
    )
    path_rules: tuple[FrameworkPathRule, ...] = ()
    unmapped_role: FrameworkAtomRole | None = None
    name: str | None = None

    canonical_schema_version: str = (
        "mdstats.framework-mapping.v2"
    )
    digest_algorithm: str = "sha256"
    digest: str = ""

```

Mapping constraints

- species_roles keys are normalized positive atomic numbers.
- atom_role_overrides keys are nonnegative global atom indices.
- role values are normalized FrameworkAtomRole members.
- path_rules are immutable, ordered, and have unique IDs.
- unmapped_role=None means every active atom must resolve explicitly.
- name is descriptive provenance and does not enter structural identity.
- all identity-bearing mappings are deeply immutable.

A mapping with no possible VERTEX role is valid as a reusable definition but building a topology from a state that resolves zero vertices must fail.

Derived properties

```

mapping.max_linker_atoms: int
mapping.allowed_rule_ids: tuple[str, ...]

```

max_linker_atoms is the maximum length of any declared linker sequence. It is zero when all rules are direct.

Symbol convenience constructor

```

FrameworkMapping.from_symbol_roles(
    species_roles: Mapping[
        str,

```



```

        FrameworkAtomRole | str,
    ],
    *,
    atom_role_overrides: Mapping[
        int,
        FrameworkAtomRole | str,
    ] | None = None,
    path_rules: Iterable[FrameworkPathRule] = (),
    unmapped_role: FrameworkAtomRole | str | None = None,
    name: str | None = None,
) -> FrameworkMapping

```

The constructor uses the package's standard chemical-symbol normalization and stores atomic-number keys.

Serialization

```

mapping.to_dict() -> dict[str, Any]
FrameworkMapping.from_dict(payload) -> FrameworkMapping

```

Serialization must be schema versioned and deterministic.

Resolved roles

ResolvedFrameworkRoles

```

@dataclass(frozen=True, slots=True)
class ResolvedFrameworkRoles:
    active_atom_indices: NDArray[np.int64]
    active_atomic_numbers: NDArray[np.int32]
    roles: tuple[FrameworkAtomRole, ...]

    vertex_atom_indices: NDArray[np.int64]
    linker_atom_indices: NDArray[np.int64]
    spectator_atom_indices: NDArray[np.int64]
    excluded_atom_indices: NDArray[np.int64]

    mapping_digest: str

```

Constraints

- active arrays exactly match the source connectivity state;
- roles has length `n_active`;
- role-specific arrays are sorted, disjoint subsets of active atoms;
- their union equals the active atom set;
- all NumPy arrays are copied, C-contiguous, and read-only;
- `mapping_digest` equals the mapping used for resolution.

Role resolution function

```

def resolve_framework_roles(
    state: AtomicConnectivityState,
    mapping: FrameworkMapping,
) -> ResolvedFrameworkRoles:
    ...

```

The function must reject:

- unsupported connectivity schema versions;
- active atoms with no resolvable role and no explicit `unmapped_role`;
- zero resolved vertices when called from `build_framework_topology`.

Overrides for atoms outside the current active state are permitted because one reusable mapping may be applied to several scoped connectivity states. Such overrides remain mapping provenance and must not be silently deleted from the mapping digest.

Projection safety options

FrameworkProjectionOptions

```
@dataclass(frozen=True, slots=True)
class FrameworkProjectionOptions:
    max_linker_atoms: int = 16
    max_candidate_paths: int = 1_000_000
    max_projected_edges: int = 1_000_000
```

These are hard safety limits, not sampling controls.

candidate_path_count increments for every lifted partial-path extension that reaches a VERTEX or LINKER atom after spectator/excluded branches have been blocked. The count therefore includes rejected prefixes and terminal paths, and is suitable as a deterministic safety diagnostic rather than a count of accepted edges.

Constraints

- max_linker_atoms is a nonnegative integer;
- the candidate-path and projected-edge limits are positive integers;
- max_linker_atoms may be zero only when every declared rule is direct;
- if mapping.max_linker_atoms exceeds the option limit, projection fails before traversal;
- exceeding a count raises FrameworkComplexityError;
- the module must never silently truncate, sample, merge, or discard accepted paths to satisfy a limit.

The options are excluded from topology identity because successful projection must be independent of a nonbinding safety ceiling.

Periodic atomic-path convention

For an oriented atomic edge $i \rightarrow j$, the source connectivity state stores an integer image shift $\mathbf{m}_{ij}$ such that the physical fractional displacement is

$$\Delta \mathbf{s}_{ij} = \mathbf{s}_j - \mathbf{s}_i + \mathbf{m}_{ij}.$$

For the reversed orientation,

$$\mathbf{m}_{ji} = -\mathbf{m}_{ij}.$$

For an oriented path

$$p = (v_0, v_1, \dots, v_k),$$

its cumulative image offsets are

$$\mathbf{q}_0 = \mathbf{0}, \quad \mathbf{q}_{\ell+1} = \mathbf{q}_\ell + \mathbf{m}_{v_\ell v_{\ell+1}}.$$

The raw projected translation is

$$\boxed{\mathbf{M}_{\text{raw}} = \mathbf{q}_k = \sum_{\ell=0}^{k-1} \mathbf{m}_{v_\ell v_{\ell+1}}}.$$

The traversal operates on lifted states

$$(v_\ell, \mathbf{q}_\ell),$$

not canonical atom indices alone. This distinction is necessary because a valid periodic path may visit another image of the same canonical atom.

Projected edge identity

FrameworkEdgeKey

```
@dataclass(frozen=True, slots=True)
class FrameworkEdgeKey:
    vertex_i: int
    vertex_j: int
    image_shift: tuple[int, int, int]

    internal_linker_indices: tuple[int, ...]
    internal_linker_image_offsets: tuple[
        tuple[int, int, int],
        ...,
    ]

    rule_id: str
```

`image_shift` is the canonical framework-gauge translation from `vertex_i` to `vertex_j`.

The internal linker image offsets are path-local offsets relative to the chosen source orientation before framework gauge normalization. They preserve distinct lifted periodic paths and remain useful for later visualization.

Key constraints

- vertex and linker indices are nonnegative global atom indices;
- `vertex_i` $\leq$ `vertex_j`;
- internal index and image-offset tuples have equal length;
- every internal atom is resolved as `LINKER`;
- internal lifted states (`atom_index`, `image_offset`) are unique;
- internal atoms cannot equal either endpoint lifted state;
- `rule_id` exists in the mapping;
- if `vertex_i` == `vertex_j`, `image_shift` must be nonzero;
- a zero-shift self-edge is invalid;
- image-shift components are zero on nonperiodic axes.

Orientation canonicalization

For distinct endpoints, the lower global atom index comes first. This canonical storage orientation is an atom-identity convention, not a physical arrow. Reversing a traversal simultaneously:

- exchanges the endpoint vertices;
- reverses the ordered atomic path;
- reverses the internal linker sequence;
- reverses and negates the atomic edge-image shifts;
- negates the projected and raw image shifts.

For a self-image edge, the implementation compares the complete forward and reverse records lexicographically and stores the smaller orientation. This prevents a periodic edge from being represented twice as $\mathbf{M}$ and $-\mathbf{M}$ while retaining an explicit traversal sign for downstream ring algorithms.

Parallel edges

Two keys may have identical endpoint atoms and identical image shifts while remaining distinct because their linker identities, lifted linker images, or rule IDs differ.

Parallel projected edges are valid and must not be collapsed.

Projected edge provenance

FrameworkEdgePath

```
@dataclass(frozen=True, slots=True)
class FrameworkEdgePath:
    key: FrameworkEdgeKey

    atomic_path_indices: tuple[int, ...]
    atomic_edge_image_shifts: tuple[
        tuple[int, int, int],
        ...,
    ]

    internal_linker_atomic_numbers: tuple[int, ...]
    raw_image_shift: tuple[int, int, int]
    edge_kind: str
```

Path constraints

For a path containing n atoms:

- `atomic_path_indices` has length $n \geq 2$;
- `atomic_edge_image_shifts` has length $n - 1$;
- path endpoints equal the key vertices in canonical orientation;
- all internal atoms equal `key.internal_linker_indices`;
- all internal roles are `LINKER`;
- no internal path state repeats in the lifted graph;
- the sum of atomic edge shifts equals `raw_image_shift`;
- the stored atomic edge shifts are expressed in the deterministic VERTEX/LINKER induced-subgraph gauge;
- internal atomic numbers align with internal atom indices;
- `edge_kind` equals the matched path rule's `edge_kind`.

`raw_image_shift` and `key.image_shift` may differ because the latter has undergone projected framework gauge normalization.

Downstream consumers must not combine raw atomic edge shifts directly with projected lifted endpoint images. They must first reconstruct the deterministic per-vertex gauge relation. `primitive_ring.py` performs this reconstruction when expanding a projected ring into a lifted atomic walk.

Oriented edge traversal view

OrientedFrameworkEdgePath

```
@dataclass(frozen=True, slots=True)
class OrientedFrameworkEdgePath:
    edge: FrameworkEdgePath
    orientation: Literal[-1, 1] = 1
```

The authoritative `FrameworkEdgePath` is stored once in canonical endpoint orientation. `OrientedFrameworkEdgePath` is a derived read-only traversal view. It does not create a second graph edge and does not alter topology identity.

```
edge.oriented(+1) -> OrientedFrameworkEdgePath
edge.oriented(-1) -> OrientedFrameworkEdgePath
edge.oriented_from(source_vertex) -> OrientedFrameworkEdgePath
```

For orientation $\eta \in \{+1, -1\}$,

$$\mathbf{M}^{(\eta)} = \eta \mathbf{M}.$$

When $\eta = -1$, the atomic path and linker sequence are reversed and every atomic edge-image shift is reversed in order and negated. Image offsets are re-expressed relative to the new traversal source.

`oriented_from()` is valid only for non-self edges. A self-image edge has the same canonical endpoint atom at both ends and therefore requires an explicit orientation sign.

This view is the required interface for primitive-ring traversal and oriented ring-edge incidence. The projected graph remains an undirected multigraph.

Projection report

FrameworkProjectionReport

```
@dataclass(frozen=True, slots=True)
class FrameworkProjectionReport:
    role_counts: Mapping[FrameworkAtomRole, int]

    linker_atom_indices: NDArray[np.int64]
    linker_framework_degree: NDArray[np.int32]
    linker_used: NDArray[np.bool_]

    candidate_path_count: int
    accepted_edge_count: int
    duplicate_path_count: int
    ignored_atomic_edge_count: int

    parallel_vertex_pair_count: int
    self_image_edge_count: int
```

`linker_framework_degree` counts source atomic edges from each linker to atoms resolved as VERTEX or LINKER. Edges to spectators and excluded atoms do not contribute.

Derived diagnostics

```
report.unused_linker_atom_indices
report.dangling_linker_atom_indices
report.branching_linker_atom_indices
```

with

$$\begin{aligned} \text{unused} &: \text{linker appears in no accepted edge,} \\ \text{dangling} &: d_{\text{framework}} < 2, \\ \text{branching} &: d_{\text{framework}} > 2. \end{aligned}$$

These conditions are diagnostics, not automatic errors. A defect study may need to preserve them exactly.

`ignored_atomic_edge_count` counts atomic edges that cannot participate because at least one endpoint is a spectator or excluded atom.

Framework topology object

FrameworkTopology

```
@dataclass(frozen=True, slots=True)
class FrameworkTopology:
    vertex_atom_indices: NDArray[np.int64]
    vertex_atomic_numbers: NDArray[np.int32]
    pbc: NDArray[np.bool_]

    edges: tuple[FrameworkEdgePath, ...]

    degree: NDArray[np.int32]
    component_labels: NDArray[np.int32]
    n_components: int

    resolved_roles: ResolvedFrameworkRoles
```

```

projection_report: FrameworkProjectionReport
validation: FrameworkValidationReport | None

source_connectivity_digest: str
mapping_digest: str

canonical_schema_version: str = (
    "mdstats.framework-topology.v2"
)
digest_algorithm: str = "sha256"
graph_digest: str = ""
digest: str = ""

```

Array shapes

Field	Shape
vertex_atom_indices	(n_vertices,)
vertex_atomic_numbers	(n_vertices,)
pbcc	(3,)
degree	(n_vertices,)
component_labels	(n_vertices,)

All arrays are copied, C-contiguous, and read-only.

Topology constraints

- at least one framework vertex is required;
- vertex indices are strictly increasing;
- atomic numbers align with vertex indices;
- every edge endpoint belongs to the vertex set;
- edge records are canonical and lexicographically sorted;
- exact duplicate edge keys are forbidden;
- parallel distinct edge keys are allowed;
- degree and components are recomputed and verified;
- component labels are deterministic and use dense labels starting at zero;
- n_components >= 1;
- disconnected and broken frameworks are valid;
- validation results do not change structural identity.

Degree convention

A normal projected edge contributes one to each endpoint degree. A self-image edge contributes two to the degree of its single canonical endpoint, consistent with multigraph loop degree:

$$d_i = \sum_{e \ni i} \begin{cases} 1, & e \text{ joins distinct vertices,} \\ 2, & e \text{ is a self-image edge at } i. \end{cases}$$

Components

Components are calculated on the quotient vertex multigraph while ignoring edge translation. Parallel edges do not change component membership. Self-image edges do not connect different canonical vertices.

Convenience interface

```

topology.n_vertices
topology.n_edges
topology.edge_keys
topology.degree_for_atom(atom_index)

```

```

topology.edges_for_atom(atom_index)
topology.to_dict()
FrameworkTopology.from_dict(payload)
topology.to_networkx()

```

`to_networkx()` returns a derived `networkx.MultiGraph`. It is not authoritative for identity or serialization.

Role-constrained projection algorithm

Step 1: validate and resolve roles

1. Validate the input `AtomicConnectivityState`.
2. Normalize and validate the `FrameworkMapping`.
3. Resolve one role for every active atom.
4. Require at least one VERTEX atom.
5. Build role-specific sorted atom arrays.

Step 2: construct oriented periodic adjacency

For each canonical atomic edge

$$(i, j, \mathbf{m}_{ij}), \quad i < j,$$

create two adjacency records:

$$i \rightarrow j : \mathbf{m}_{ij}, \quad j \rightarrow i : -\mathbf{m}_{ij}.$$

Adjacency lists must be sorted deterministically by neighbor atom index and image shift.

Before path traversal, the implementation re-normalizes the induced atomic subgraph containing only atoms resolved as VERTEX or LINKER. Spectator and excluded edges are not used to choose this gauge. This is essential because the canonical gauge of the full atomic graph may change when spectator contacts are added, even though the physical framework is unchanged.

For a framework-relevant atomic edge $i \rightarrow j$, the induced-subgraph gauge uses

$$\tilde{\mathbf{m}}_{ij} = \mathbf{m}_{ij} + \mathbf{h}_i - \mathbf{h}_j,$$

where $\mathbf{h}_i$ is obtained from a deterministic spanning forest of the VERTEX/LINKER induced graph. Traversal, stored atomic path shifts, linker image offsets, and raw projected shifts use $\tilde{\mathbf{m}}_{ij}$. Consequently, adding or removing spectator-only contacts cannot change the projected graph or its digest.

Step 3: precompute accepted rule prefixes

The implementation constructs exact linker-sequence prefixes for every rule and its reverse. This permits early pruning.

For rules

```

(0)
(0, 0)
(0, S)

```

valid prefixes include

```

()
(0)
(0, 0)
(0, S)
(S)
(S, 0)

```

subject to endpoint-species compatibility.

Step 4: traverse the lifted atomic graph

For each source vertex in increasing global atom-index order, perform a bounded depth-first search or equivalent deterministic traversal.

Each traversal state contains:

current atom index
current cumulative image offset
full atomic path
atomic step shifts
internal linker species sequence
visited lifted states

For every adjacent atom:

- SPECTATOR: stop that branch;
- EXCLUDED: stop that branch;
- LINKER: continue only when the sequence is an accepted rule prefix;
- VERTEX: stop and test the completed path against exact rules.

A vertex encountered inside a path terminates the branch. The algorithm must not skip an intermediate framework vertex to create a longer edge.

The path is simple in the lifted graph:

$$(v_a, \mathbf{q}_a) \neq (v_b, \mathbf{q}_b) \quad \text{for } a \neq b,$$

except that the terminal state may be another periodic image of the source vertex.

Step 5: accept terminal paths

A terminal path is accepted only when:

1. every internal atom is LINKER;
2. its complete endpoint/linker/endpoint signature matches exactly one path rule or the complete reverse of that rule;
3. endpoint and linker orientation are evaluated together;
4. the internal linker count satisfies the safety limit;
5. a source-to-source path has nonzero total translation;
6. the projected edge limit is not exceeded.

The algorithm records all valid branches. Branching linkers and multiple routes are not resolved by choosing a shortest or first path.

Step 6: canonicalize path orientation

For distinct endpoints, orient the path from lower to higher global atom index. For self-image paths, choose the lexicographically smaller forward or reverse record.

Canonicalize internal linker identities and path-local image offsets with the same orientation.

Step 7: deduplicate exact path records

Reverse discovery and repeated traversal may rediscover the same physical path. Exact canonical edge records are deduplicated.

Distinct paths are retained when they differ by:

- internal linker atom identity;
- lifted linker image;
- total periodic translation;
- matched rule ID;
- ordered linker identity relative to canonical endpoint orientation.

Step 8: normalize the projected periodic gauge

The raw projected graph has vertex-gauge freedom. If framework vertex representatives change by integer offsets $\mathbf{g}_i$, projected shifts transform as

$$\mathbf{M}'_{ij} = \mathbf{M}_{ij} + \mathbf{g}_i - \mathbf{g}_j.$$

For each projected component:

1. choose the smallest framework atom index as root;
2. set its gauge to zero;
3. build a deterministic spanning tree from sorted provisional edge records;
4. ignore self-image edges when choosing tree links;
5. propagate gauges along tree edges;
6. transform every projected edge shift;
7. retain non-tree winding translations;
8. sort the final canonical edge records.

All tree edges between distinct vertices become zero-shift. Parallel non-tree edges and self-image edges retain periodic cycle information.

Step 9: calculate graph observables and identity

Calculate:

- multigraph degree;
- deterministic connected components;
- projection diagnostics;
- graph-only structural digest;
- mapping-aware topology digest;
- optional validation report.

Pseudocode

```
function build_framework_topology(state, mapping, rules=None, options=None):
    validate state, mapping, options
    resolved = resolve_framework_roles(state, mapping)
    require at least one vertex

    adjacency = oriented_periodic_adjacency(state)
    matcher = compile_exact_rule_prefixes(mapping.path_rules)

    provisional = empty list
    report counters = zero

    for source in sorted(resolved.vertex_atom_indices):
        stack = initial lifted traversal state at source

        while stack is not empty:
            path_state = pop deterministic next state

            for neighbor, step_shift in adjacency[current]:
                next_image = current_image + step_shift
                lifted = (neighbor, next_image)

                if lifted repeats an internal path state:
                    continue

                role = resolved.role_of(neighbor)

                if role is SPECTATOR or EXCLUDED:
```

```

        count ignored branch
        continue

    if role is LINKER:
        next_sequence = sequence + atomic_number(neighbor)
        if next_sequence is not a valid rule prefix:
            continue
        enforce path limits
        push extended state
        continue

    assert role is VERTEX
    count candidate path
    match exact rule using endpoint species and sequence
    if no exact rule:
        continue
    if neighbor == source and next_image == (0, 0, 0):
        continue

    canonical_path = orient_path_canonically(...)
    provisional.append(canonical_path)

deduplicate exact provisional paths
normalize projected vertex gauge
sort canonical edge records
calculate degree and components
build projection report
calculate graph and topology digests

topology = FrameworkTopology(...)

if validation rules were supplied:
    report = validate_framework_topology(topology, rules)
    attach report without changing topology identity

return topology

```

Complexity

Let:

- N be the number of active atoms;
- E be the number of atomic edges;
- N_V be the number of framework vertices;
- L be the maximum allowed linker count;
- b be the effective linker-graph branching factor;
- P be the number of candidate terminal paths;
- K be the number of accepted projected edges.

Role resolution and adjacency construction are

$$O(N + E).$$

Bounded path discovery is data dependent. Its worst-case search count scales as

$$\left(\rho N_V \sum_{\ell=0}^L b^\ell \right),$$

which is why exact prefix pruning and hard candidate limits are required.

Canonicalization and sorting are approximately

$$O(K \log K),$$

excluding total variable path-record length.

No algorithm may claim polynomial behavior for arbitrary branching linker graphs with user-defined path depth. The implementation must expose and enforce its safety limits.

Structural identity and digests

Define

```
CANONICAL_FRAMEWORK_MAPPING_SCHEMA = (
    "mdstats.framework-mapping.v2"
)
CANONICAL_FRAMEWORK_TOPOLOGY_SCHEMA = (
    "mdstats.framework-topology.v2"
)
FRAMEWORK_DIGEST_ALGORITHM = "sha256"
```

Mapping digest

The mapping digest includes:

- mapping schema;
- normalized species roles;
- normalized atom overrides;
- normalized path rules;
- explicit unmapped role.

It excludes the descriptive mapping name.

Graph digest

graph_digest includes:

- topology schema;
- pbc;
- sorted vertex atom indices;
- aligned vertex atomic numbers;
- sorted canonical projected edge keys.

It excludes:

- Cartesian coordinates;
- cell lengths and strain;
- source frame index;
- source connectivity digest;
- projection limits;
- diagnostic counts;
- validation results;
- visualization metadata.

Mapping-aware topology digest

digest includes:

- graph_digest;
- mapping_digest.

The source connectivity digest is stored as provenance but is not included in the projected graph digest. Atomic spectator contacts that do not affect projection must therefore leave graph_digest unchanged.

Two topology objects are equal only when their schemas, normalized mappings, vertex records, and canonical decorated edge keys are exactly equal. General unlabeled graph isomorphism is not the equality definition.

Validation API

FrameworkValidationRules

```
@dataclass(frozen=True, slots=True)
class FrameworkValidationRules:
    allowed_vertex_degrees: Mapping[
        int,
        frozenset[int],
    ] = field(default_factory=dict)

    allowed_linker_degrees: Mapping[
        int,
        frozenset[int],
    ] = field(default_factory=dict)

    expected_vertex_count: int | None = None
    expected_edge_count: int | None = None

    require_single_component: bool = False
    require_all_linkers_used: bool = False
    allow_parallel_edges: bool = True
    allow_self_image_edges: bool = True

    allowed_edge_kinds: frozenset[str] | None = None
```

Degree mappings are keyed by atomic number. A missing species key is unconstrained.

allowed_linker_degrees refers to framework-relevant atomic degree, counting only source atomic edges to VERTEX or LINKER atoms.

FrameworkValidationIssue

```
@dataclass(frozen=True, slots=True)
class FrameworkValidationIssue:
    code: str
    severity: Literal["warning", "error"]
    message: str
    atom_indices: tuple[int, ...] = ()
    edge_indices: tuple[int, ...] = ()
```

Issue codes must be stable machine-readable strings, for example:

```
unexpected_vertex_count
unexpected_edge_count
invalid_vertex_degree
invalid_linker_degree
disconnected_framework
unused_linker
parallel_edge_not_allowed
self_image_edge_not_allowed
unexpected_edge_kind
```

FrameworkValidationReport

```
@dataclass(frozen=True, slots=True)
class FrameworkValidationReport:
    rules: FrameworkValidationRules
    issues: tuple[FrameworkValidationIssue, ...]
    summary_counts: Mapping[str, int]

    @property
    def passed(self) -> bool: ...
```

```
def raise_for_errors(self) -> None: ...
```

passed is true when no issue has severity "error".

`raise_for_errors()` raises `FrameworkValidationError` containing a concise summary. Validation is otherwise nonthrowing and nonmutating.

Validation function

```
def validate_framework_topology(  
    topology: FrameworkTopology,  
    rules: FrameworkValidationRules,  
) -> FrameworkValidationReport:  
    ...
```

Validation must not:

- remove an edge;
- reassign a role;
- merge parallel paths;
- reconnect a fragment;
- suppress an unused linker;
- alter any digest.

Primary build function

```
def build_framework_topology(  
    state: AtomicConnectivityState,  
    mapping: FrameworkMapping,  
    *,  
    validation_rules: FrameworkValidationRules | None = None,  
    options: FrameworkProjectionOptions | None = None,  
) -> FrameworkTopology:  
    ...
```

Inputs

state

One immutable canonical `AtomicConnectivityState`.

Required properties:

- nonempty active atom set;
- valid atomic numbers;
- valid periodic flags;
- canonical sorted atomic edges;
- one atomic edge per unordered atom pair, matching the current connectivity module limitation;
- image shifts zero on nonperiodic axes.

mapping

One immutable normalized `FrameworkMapping`.

validation_rules

Optional material-specific checks. If absent,

`topology.validation` is `None`

options

Optional projection safety limits. Defaults are used when absent.

Output

One immutable FrameworkTopology representing the exact projected graph for the input connectivity state and mapping.

Failure modes

- TypeError: unsupported input object type;
- FrameworkMappingError: unresolved roles, overlapping rules, invalid mapping;
- FrameworkProjectionError: invalid path or periodic projection invariant;
- FrameworkComplexityError: declared safety limit exceeded;
- FrameworkTopologyError: inconsistent final topology object;
- FrameworkValidationError: only when the caller explicitly invokes `raise_for_errors()`.

LTA zeolite reference mapping

For the relaxed Na-LTA integration fixture:

```
mapping = FrameworkMapping.from_symbol_roles(
    {
        "Si": FrameworkAtomRole.VERTEX,
        "Al": FrameworkAtomRole.VERTEX,
        "O": FrameworkAtomRole.LINKER,
        "Na": FrameworkAtomRole.SPECTATOR,
    },
    path_rules=(
        FrameworkPathRule.from_symbols(
            rule_id="T-O-T",
            linker_symbols=("O",),
            edge_kind="oxygen_bridge",
        ),
    ),
    name="Na-LTA T-site framework",
)
```

Recommended validation is

```
rules = FrameworkValidationRules(
    allowed_vertex_degrees={
        14: frozenset({4}), # Si
        13: frozenset({4}), # Al
    },
    allowed_linker_degrees={
        8: frozenset({2}), # O
    },
    expected_vertex_count=48,
    expected_edge_count=96,
    require_single_component=True,
    require_all_linkers_used=True,
    allow_parallel_edges=False,
    allow_self_image_edges=False,
    allowed_edge_kinds=frozenset({"oxygen_bridge"}),
)
```

For the supplied relaxed primitive-cell structure, the expected framework result is

```
48 projected T vertices
96 projected T-T edges
24 Si vertices
24 Al vertices
96 linker O atoms
all T-site degrees equal to 4
all linker framework degrees equal to 2
one projected component
```

The degree identity is

$$48 \times 4 = 2 \times 96 = 192.$$

A broader atomic connectivity state may also contain Na-O contacts. Because Na is resolved as SPECTATOR, those edges must not change the projected topology, graph_digest, or the 48/96 counts.

Edge cases and required behavior

Unmapped active species

With `unmapped_role=None`, any active atom lacking a species role and atom override is an error. This prevents silent omission of an unexpected species.

Users may explicitly choose

```
unmapped_role=FrameworkAtomRole.EXCLUDED
```

when broad exclusion is scientifically intended.

Same species in several roles

Species-level mapping is insufficient when the same element appears in the framework and outside it. Explicit atom-index overrides are required. The module must not infer roles from coordination automatically.

Spectator contacts

A path may not pass through a spectator even when atomic connectivity contains short contacts such as

```
Si - O - Na - O - Al
```

Na-O contacts may be analyzed elsewhere but cannot bridge framework vertices.

Intermediate framework vertices

Traversal stops at the first encountered VERTEX. A path such as

```
Si - O - Al - O - Si
```

produces two candidate edges, not one long Si-Si edge.

Terminal and dangling linkers

A linker attached to only one framework-relevant neighbor remains in diagnostics but produces no projected edge unless an exact terminal rule is introduced in a future directed or hypergraph extension.

Branching linkers

A branching linker may produce several valid paths. The first release preserves all exact accepted paths and reports the branching degree. Validation may reject it for materials that require two-connected bridges.

Linker-only cycles

A linker-only cycle not connected to two accepted framework vertices produces no projected edge. Its linker atoms appear as unused linkers.

Parallel paths

Two O bridges connecting the same T atoms are separate edges. Ring algorithms must receive the multigraph rather than a silently simplified graph.

Self-image edges

A projected path may connect a vertex to another periodic image of itself. Such an edge is valid only with nonzero translation. It contributes degree two and must be preserved for periodic topology.

Zero-shift self-return

A closed path returning to the same canonical vertex with zero translation is not a valid projected edge. It is a cycle-like structure requiring a different representation and must be rejected or recorded as a projection diagnostic.

Repeated canonical atoms in different images

Path simplicity is evaluated on (atom_index, image_offset), not atom index alone. Repetition of one canonical atom in a different periodic image may be valid within the configured depth.

Asymmetric linker sequences

A bridge such as

A - 0 - S - B

is equivalent only to its complete reverse

B - S - 0 - A.

It is distinct from A-S-0-B. The graph remains undirected, but every projected edge preserves an ordered canonical path and exposes an explicit reverse traversal view. Any simplification that stores only an unordered endpoint pair and an independently reversal-canonical linker multiset is invalid.

Overlapping rules

A path must match exactly one rule. Mapping construction rejects overlapping rule domains rather than relying on declaration order.

Excessive path growth

Highly branched linker networks may produce exponential candidate growth. The module raises `FrameworkComplexityError` at the configured limit and returns no partial topology.

Disconnected or broken frameworks

Disconnected components, isolated vertices, missing bridges, and unused linkers are valid topology representations. They become validation issues only when the caller requests corresponding rules.

Strain and cell rotation

Topology identity does not depend on Cartesian cell orientation, QR form, strain, or lattice-vector lengths. Only periodic flags and canonical integer edge translations enter the graph identity.

Atomic connectivity model changes

Different cutoff or hysteresis definitions may produce different source atomic states and therefore different framework topologies. The topology module records the source connectivity digest but does not judge which connectivity definition is scientifically correct.

Pseudoelements and nonstandard atomic numbers

The first release assumes standard positive atomic numbers. Dummy atoms, pseudoelements, coarse-grained sites, and atomic number zero require a future explicit species-identity abstraction.

Serialization contract

Every public identity-bearing object should support schema-versioned dictionary serialization.

Required round trips:

```
FrameworkPathRule.from_dict(rule.to_dict()) == rule
FrameworkMapping.from_dict(mapping.to_dict()) == mapping
FrameworkEdgeKey.from_dict(key.to_dict()) == key
FrameworkEdgePath.from_dict(path.to_dict()) == path
FrameworkTopology.from_dict(topology.to_dict()) == topology
```

JSON-compatible payloads must use:

- lists for tuples and arrays;
- strings for enum values;
- decimal integers for atom indices and shifts;
- explicit schema and digest fields;

- deterministic key ordering when encoded for hashing.

Deserialization must recompute and verify digests rather than trusting stored hash strings.

NetworkX conversion

`topology.to_networkx()` -> `networkx.MultiGraph`

Recommended node attributes:

```
atom_index
atomic_number
symbol
degree
component_id
```

Recommended edge attributes:

```
edge_index
rule_id
edge_kind
image_shift
raw_image_shift
internal_linker_indices
internal_linker_atomic_numbers
atomic_path_indices
reverse_atomic_path_indices
atomic_edge_image_shifts
reverse_atomic_edge_image_shifts
orientation_aware
canonical_orientation
```

NetworkX is a convenience and interoperability layer. NetworkX node order, iteration order, object hashes, and graph serialization must not define mdstats structural identity.

Implemented work plan

F2.1 - role and mapping foundation

Implemented:

- exception hierarchy;
- `FrameworkAtomRole`;
- atomic-number and symbol normalization;
- `FrameworkPathRule` with coupled whole-path signatures;
- whole-path reversal canonicalization and overlap detection;
- `FrameworkMapping`;
- stable mapping serialization and digest;
- `ResolvedFrameworkRoles`;
- `resolve_framework_roles()`.

Acceptance criteria:

- role precedence is exact;
- mappings are deeply immutable;
- symbol and atomic-number constructors agree;
- overlapping rules fail deterministically;
- serialization round trips preserve digests.

F2.2 - periodic path projection

Implemented:

- oriented adjacency from `AtomicConnectivityState`;

- exact rule-prefix matcher;
- deterministic lifted-state traversal;
- spectator and excluded blocking;
- terminal vertex handling;
- direct-edge rules;
- periodic translation accumulation;
- self-image path support;
- path canonicalization and exact deduplication;
- hard complexity limits.

Acceptance criteria:

- no bonds are rebuilt from coordinates;
- path order is deterministic;
- reverse discovery gives one canonical edge;
- asymmetric endpoint/linker signatures retain their orientation coupling;
- derived oriented traversal views reverse all path and periodic data together;
- parallel distinct paths survive;
- no partial result is returned after a limit error.

F2.3 - projected graph canonicalization

Implemented:

- FrameworkEdgeKey;
- FrameworkEdgePath;
- deterministic projected spanning forest;
- framework gauge normalization;
- multigraph degree;
- deterministic components;
- stable sorted edge ordering;
- graph and mapping-aware digests;
- FrameworkTopology.

Acceptance criteria:

- independent atomic wrapping produces equal topology;
- non-tree winding is retained;
- self-image edges remain nonzero;
- exact equality is traversal-order independent;
- coordinates and cell rotation do not alter digests.

F2.4 - diagnostics and validation

Implemented:

- FrameworkProjectionReport;
- linker framework degrees;
- unused, dangling, and branching linker diagnostics;
- FrameworkValidationRules;
- issue and report objects;
- `validate_framework_topology()`;
- `explicit_raise_for_errors()`.

Acceptance criteria:

- validation never modifies topology;
- disconnected frameworks remain constructible;
- every issue has a stable code and affected indices;

- validation report serialization is deterministic.

F2.5 - interoperability and documentation

Implemented:

- public exports;
- dictionary round trips;
- NetworkX MultiGraph conversion;
- API docstrings;
- minimal usage examples;
- package version update;
- source/specification consistency audit.

F2.6 - Na-LTA integration

Use the supplied relaxed Na-LTA POSCAR as the primary system test.

Run two source-connectivity cases:

1. framework-only Si/Al/O connectivity;
2. broader connectivity containing illustrative Na-O contacts.

Both must project to the same framework graph:

```
48 vertices
96 edges
all T degrees 4
one component
```

The broader graph must record spectator-related ignored atomic edges without changing the projected `graph_digest`.

Test matrix

The implementation must include focused tests for:

1. one simple vertex-linker-vertex path;
2. one direct vertex-vertex rule;
3. absent direct rule rejection;
4. symbol and atomic-number mapping equivalence;
5. atom-role override precedence;
6. unmapped active species error;
7. spectator blocking;
8. excluded-atom blocking;
9. intermediate vertex termination;
10. dangling linker diagnostics;
11. branching linker diagnostics;
12. parallel paths through different linkers;
13. exact reverse-path deduplication;
14. distinct periodic paths through different linker images;
15. periodic shift composition;
16. gauge invariance under independent wrapping;
17. projected self-image edge;
18. zero-shift self-return rejection;
19. disconnected projected components;
20. isolated framework vertex preservation;
21. overlapping rule rejection;
22. candidate-path complexity failure;
23. projected-edge complexity failure;
24. serialization and digest round trips;

25. NetworkX multigraph conversion;
26. validation success and failure reports;
27. relaxed Na-LTA 48/96 acceptance;
28. Na spectator-contact invariance.

Implementation status

The mdstats 0.15.0 Stage 2 implementation satisfies this specification with:

- immutable role, mapping, edge, report, validation, and topology objects;
- exact lifted-path traversal with hard complexity ceilings;
- induced-subgraph gauge normalization independent of spectator contacts;
- deterministic projected multigraph gauge normalization;
- dictionary round trips and NetworkX MultiGraph conversion;
- relaxed Na-LTA acceptance and spectator-contact invariance tests;
- full-package regression coverage.

Acceptance criteria for the module

The Stage 2 implementation is complete only when:

1. the public API matches this specification;
2. all identity-bearing objects are immutable;
3. no coordinate-based bond reconstruction occurs;
4. all accepted projected edges retain atomic-path provenance;
5. spectators and excluded atoms cannot alter framework connectivity;
6. parallel paths and nonzero self-image edges are preserved;
7. projected periodic gauge is deterministic;
8. topology equality is independent of traversal order and atom wrapping;
9. path explosion fails explicitly without truncation;
10. validation reports defects without repairing them;
11. Na-LTA produces 48 T vertices and 96 T-T edges;
12. broader Na-O connectivity leaves the LTA projected graph unchanged;
13. Markdown and PDF specifications match the implemented source;
14. the complete package regression suite remains clean.

Future extensions

The first release intentionally defers:

- automatic smoothing or probabilistic reconciliation of topology catalogs;
- transition kinetics or chemical event naming;
- intrinsically directed physical relations such as reaction or transport arrows;
- hyperedges involving more than two retained vertices;
- terminal functional-group topology;
- bond-order-weighted paths;
- probabilistic role assignment;
- pseudoelement identity;
- primitive-ring enumeration;
- ring geometry and site classification;
- cage construction;
- topology-aware dynamic regions.

Stage 3 `topology_catalog.py` is implemented in mdstats 0.16.0 and consumes the canonical structural edge keys defined here. The next scientific module is

`primitive_ring.py`

using the canonical decorated periodic multigraph defined here.

Final architectural invariant

The module must preserve the separation

$$\boxed{\text{atomic connectivity} \neq \text{framework topology} \neq \text{primitive rings} \neq \text{ring geometry}} .$$

Each layer adds one explicit scientific interpretation. No later abstraction may silently rewrite the evidence owned by an earlier layer.